LAB 07 — Software Integrity Verification

SHA256 checksums and GPG digital signatures

Student: Nenad Damjanović | Date: 30 October 2026

Objective

Inspect the provided installation script, calculate and validate a SHA256 checksum, authenticate the checksum using a detached GPG signature, simulate tampering, and verify before execution.

Environment: Kali Linux, GnuPG 2.4.9, instructor demo install.sh.

01 — Script inspection

The provided install.sh was inspected before execution. It prints an installation message, creates src/, writes a small run.sh script, moves it into src/, and runs it to print Hello. It is a demonstration rather than a real software installer.

```
cat install.sh
ls -l install.sh
file install.sh
```

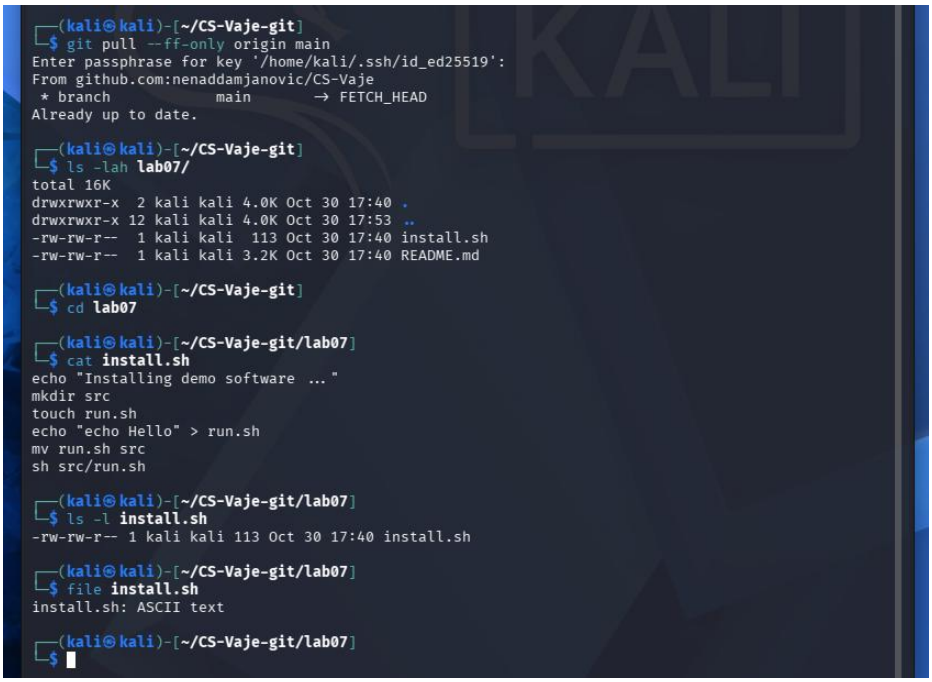


Figure 1. 01-script-inspection.png

02 — SHA256 checksum

SHA256 was calculated for the original script, recorded in install.sh.sha256, and checked successfully. This confirms the file matches the recorded digest; it does not establish who created the digest.

```
sha256sum install.sh > install.sh.sha256
cat install.sh.sha256
sha256sum -c install.sh.sha256
# install.sh: OK
```

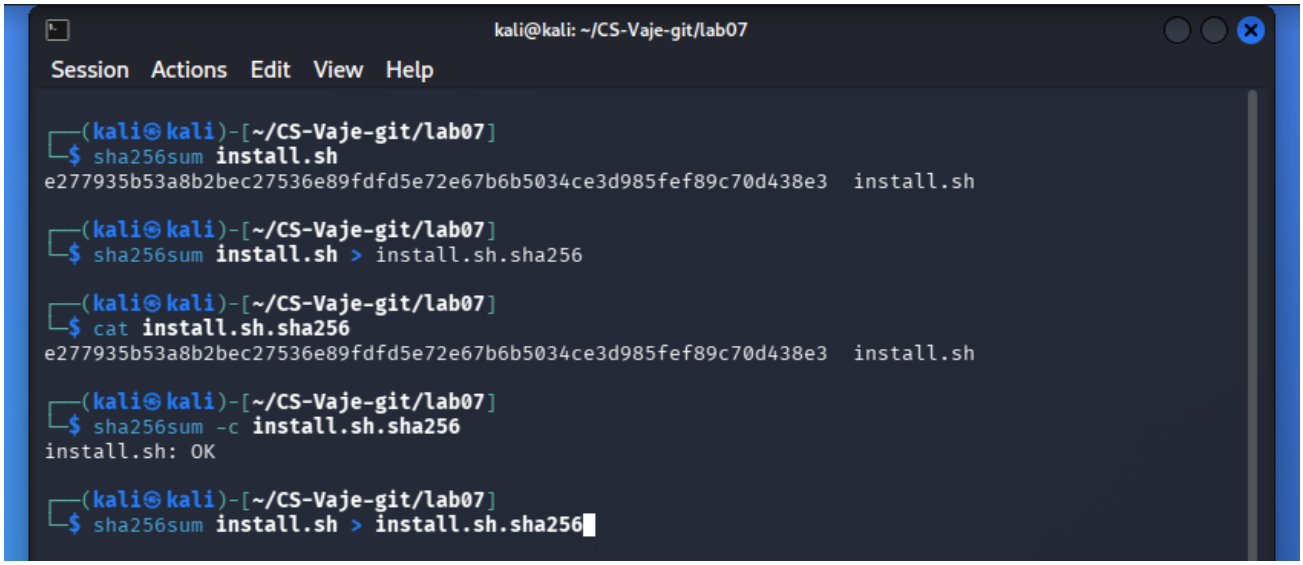


Figure 2. 02-sha256-verification.png

03 — Detecting script modification

A separate copy, install_modified.sh, was changed by appending a warning command. Its SHA256 digest differed from the original and the checksum verification reported FAILED. In the screenshot, the original script lacked a trailing newline, so the appended text joined the existing last line; the modified copy was not executed.

```
cp install.sh install_modified.sh
echo 'echo "WARNING: Script has been modified!"' >> install_modified.sh
sha256sum install.sh install_modified.sh
sed 's/install\.sh$/install_modified.sh/' install.sh.sha256 | sha256sum -c
# install_modified.sh: FAILED
```

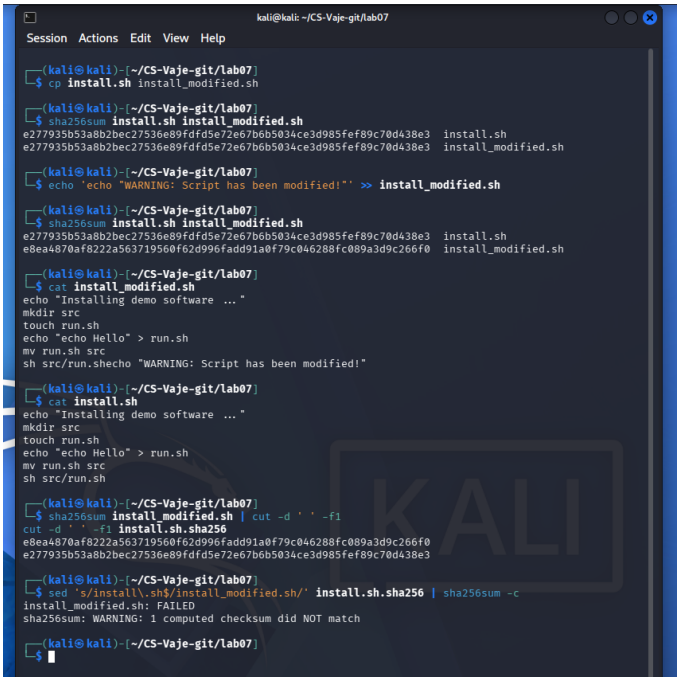


Figure 3. 03-sha256-tampering.png

04 — Signing and verifying the checksum

The already available Ed25519 secret key, Lab 06 Secrets <lab06@example.com>, was used to produce an ASCII-armored detached signature of the checksum. Signature verification reported Good signature, while SHA256 reported install.sh: OK.

```
gpg --armor --detach-sign --local-user lab06@example.com --output install.sh.sha256.asc install.sh.sha256
gpg --verify install.sh.sha256.asc install.sh.sha256
sha256sum -c install.sh.sha256
```



Figure 4. 04-gpg-signature-verification.png

05 — Detecting checksum tampering

The checksum file was copied and its digest replaced with the digest of the modified script. The original detached signature still verified against the unmodified checksum, but verification against checksum_modified.sha256 returned BAD signature. This demonstrates protection of the checksum file.

```
gpg --verify install.sh.sha256.asc install.sh.sha256
gpg --verify install.sh.sha256.asc checksum_modified.sha256
# Good signature / BAD signature
```

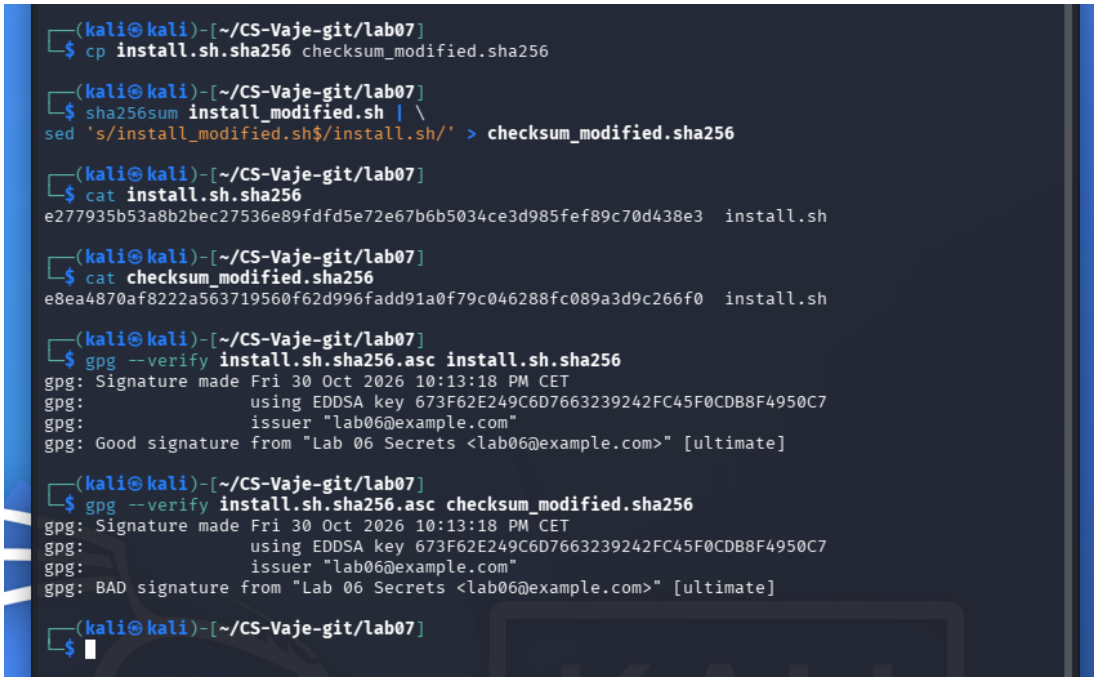


Figure 5. 05-gpg-tampering-detection.png

06 — Verify before execution

The genuine checksum signature and SHA256 digest were validated before running the original demo installer without sudo. The script printed Installing demo software ... and Hello; src/run.sh contained echo Hello. A valid signature is meaningful only if the signer's public key is independently authenticated.

```
gpg --verify install.sh.sha256.asc install.sh.sha256
sha256sum -c install.sh.sha256
sh install.sh
ls -lah src/
cat src/run.sh
```

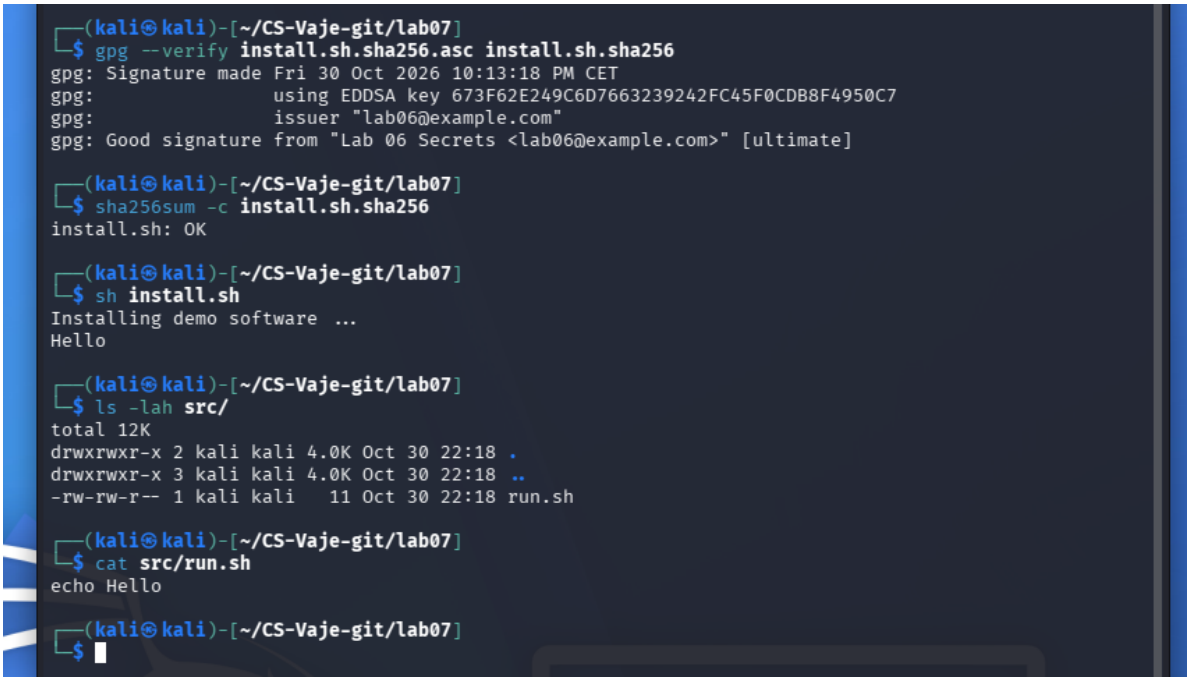


Figure 6. 06-verify-before-execute.png

Questions and answers

1. Why should you never execute scripts without inspection?

Unreviewed scripts can modify the system, download other programs, steal data, or execute harmful commands. Inspecting code reduces but does not eliminate this risk.

2. What does SHA256 guarantee?

SHA256 produces a cryptographic digest used to detect whether content differs from an expected value. A matching independently trusted digest provides strong evidence of integrity, not authorship.

3. What does GPG guarantee?

A verified digital signature binds the signed data to the holder of the corresponding private key and detects post-signing changes. Authentic identity additionally depends on verifying the public key.

4. Can SHA256 prove the author of a file?

No. Anybody can compute a checksum, including an attacker who replaces both the script and checksum.

5. Why is signing the checksum important?

A detached signature prevents undetected modification of the checksum unless the attacker can also forge a valid signature from the trusted signer.

6. What happened after the script was modified?

The modified script had a different SHA256 digest, and verification against the original checksum failed. Separately, changing the checksum file caused GPG to report BAD signature.

Security notes and conclusion

Good signature means the data match the public key used for verification. The key's identity still needs independent verification. SHA256 detects modification relative to a trusted digest, but cannot prove authorship. Neither check proves that a script is harmless. No private keys or passphrases are included.

The local demonstrations confirmed script integrity, checksum authenticity, failed verification of tampered inputs, and successful execution of the reviewed original script.

Source: Instructor materials, Verifying Software Integrity (SHA256 & GPG), Lab 07.

**ChatGPT is used to create a report from my own work and screenshots*